



Streams in Java

J.JANANI

B.SC(AI&ML)

TU6253210211023

What is Java Stream?

A Stream is a sequence of elements from a source that supports aggregate operations. It is not a data structure; it doesn't store data. Instead, it conveys elements from a source (like a Collection, Array, or I/O channel) through a pipeline of computational steps.

Types of Java Streams

1. Input Stream:

An Input Stream is used to read data from a source (like a file, keyboard, or network)

Base class: InputStream

Example:

```
import java.io.FileInputStream;

public class InputExample {
    public static void main(String[] args) throws Exception {
        FileInputStream fis = new FileInputStream("test.txt");
        int data;

        while((data = fis.read()) != -1){
            System.out.print((char)data);
        }

        fis.close();
    }
}
```

2. Output Stream:

An Output Stream is used to write data to a destination (like a file or screen).

Base class: OutputStream

Example:

```
import java.io.FileOutputStream;

public class OutputExample {
    public static void main(String[] args) throws Exception {
        FileOutputStream fos = new FileOutputStream("test.txt");
        String text = "Hello Java";

        fos.write(text.getBytes());
        fos.close();
    }
}
```

3. Byte Stream:

Byte Streams are used to handle binary data (images, audio, video, etc.).

Classes:

FileInputStream

FileOutputStream

Example:

```
FileInputStream fis = new FileInputStream("image.jpg");  
FileOutputStream fos = new FileOutputStream("copy.jpg");
```

4. Character Stream:

Character Streams are used to handle text data.

Classes:

FileReader

FileWriter

Example:

```
import java.io.FileReader;

public class CharExample {
    public static void main(String[] args) throws Exception {
        FileReader fr = new FileReader("text.txt");
        int ch;

        while((ch = fr.read()) != -1){
            System.out.print((char)ch);
        }

        fr.close();
    }
}
```

Advantages of Streams in Java

- **Efficient Data Transfer:**

Streams allow data to be transferred **quickly and efficiently** between the source (like a file or keyboard) and the destination (like a file or screen).

- **Supports Large Data:**

Streams can handle **large amounts of data** because they process data **piece by piece** instead of loading the entire data into memory.

- **Platform Independent:**

Java streams work the same way on **different operating systems** (Windows, Linux, Mac), making programs **portable**.

- **Supports Buffering:**

Streams support **buffering**, which stores data temporarily in memory before reading or writing. This **improves performance and speed**.

Java Lab Experiment

1. Aim:

To write a Java program that takes an integer as input from the user and outputs the digits in reverse order using a while loop.

2. Algorithm:

1.Start.

2.Initialize a variable reversed = 0.

3.Accept an integer input from the user (let's call it num).

4.Looping while num is not equal to 0:

- Find the last digit of the number: $\text{digit} = \text{num} \% 10$.
- Append the digit to reversed: $\text{reversed} = (\text{reversed} * 10) + \text{digit}$.
- Remove the last digit from the original number: $\text{num} = \text{num} / 10$.

5.Print the value of reversed.

6.Stop.

3. Program Code:

```
import java.util.Scanner;

public class ReverseNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int num = sc.nextInt();
        int reversed = 0;

        while (num != 0) {
            int digit = num % 10;
            reversed = reversed * 10 + digit;
            num /= 10;
        }

        System.out.println("Reversed Number: " + reversed);
        sc.close();
    }
}
```



Output:

Enter a number: 1234

Reversed Number: 4321

Result:

The Java program to reverse a given number was written, compiled, and executed successfully.





THANK YOU !!!